

Proyecto Base de Datos

Luis Aurelio Salazar Richmond

Ingeniería en Sistemas Computacionales

Introducción a la Programación

Alexander Simón Benjamín

Universidad Tecnológica

Análisis del Diseño de la Base de Datos

Antes de comenzar a desarrollar la base de datos y escribir las instrucciones en SQL, realicé un análisis del diagrama entidad y su relación. Este paso fue importante para comprender cómo debía organizar la información y cómo se relacionan los diferentes datos dentro del sistema de gestión de las reparaciones. El objetivo fue diseñar una base de datos ordenada, eficiente y que evitara la duplicación innecesaria de información.

Como primer paso, identifiqué las seis entidades principales del sistema: Usuarios, Equipos, Técnicos, Reparaciones, Asignaciones y Detalle Reparación. Posteriormente, analicé los atributos de cada tabla para determinar qué información debía almacenarse y seleccioné el tipo de dato más adecuado para cada campo, utilizando INT para los identificadores, VARCHAR para los datos de texto y DATETIME para el registro de fechas.

Luego definí las llaves primarias de cada tabla para identificar de forma única cada registro y establecí las llaves foráneas que permiten relacionar las tablas entre sí. Estas relaciones siguen la lógica del sistema, un usuario puede registrar varios equipos, cada equipo puede tener una o más reparaciones, las reparaciones son asignadas a un técnico y cada reparación tiene con un detalle donde se puede registrar el trabajo realizado.

Finalmente, se aplican restricciones como NOT NULL y UNIQUE para garantizar la integridad de la información y evitar registros incompletos o duplicados. Con este análisis previo fue posible diseñar una base de datos relacional organizada, segura y fácil de mantener, facilitando el almacenamiento y la consulta de la información del sistema.

Una vez que obtuve esta información empecé a programar la Base de Datos SQL Server, luego seguí el paso a paso de ir creando las tablas con los videos de clase y también con las dos practicas ejecutadas anteriormente me sirvieron como guía y ver el Diagrama enviado me permitió realizar este trabajo. Cabe mencionar que al final adjunte el Screenshot de mi Diagrama, sin embargo, cuando lo cree lo era igual y me percate que se puede editar y trate de acomodarlo de la mejor manera al del profesor.

Pregunta de Análisis: ¿Por qué es mejor el modelo relacional que un único archivo?

Este modelo relacional es más eficiente porque puede organizar la información en diferentes tablas y relacionarlas entre sí mediante llaves primarias y foráneas, esto evita la redundancia en la información. Si toda la información se almacenara en un único archivo, sería necesario repetir constantemente datos como el nombre del usuario, la información del equipo o del técnico cada vez que se registra una reparación. Esto aumentaría el tamaño de la base de datos y podría generar inconsistencias cuando un dato deba actualizarse en varios lugares.

Al utilizar este modelo relacional, cada dato se almacena una sola vez y las tablas se relacionan entre sí cuando es necesario. Esto facilita el mantenimiento de la información, mejora el rendimiento de las consultas, reduce el riesgo de errores y garantiza una mayor integridad de los datos. Además, permite que el sistema sea más organizado, limpio y fácil de administrar conforme vaya aumentando la cantidad de registros.

Fase 1: Investigación y Análisis del Diseño

1. Mapeo de Atributos y Tipos de Datos

Tabla	Campo	Tipo de dato	Restricción	Justificación
USUARIOS	USUARIOID	INT IDENTITY (1,1)	PRIMARY KEY	Identificador único del usuario.
	NOMBRE	VARCHAR (50)	NOT NULL	Longitud suficiente para nombres completos, evita desperdicio de espacio.
	CORREOELECTRONICO	VARCHAR (100)	NOT NULL, UNIQUE	Correos pueden ser largos; la restricción evita duplicados.
	TELEFONO	VARCHAR (20)	-	Admite caracteres especiales y números de diferentes formatos.
EQUIPOS	EQUIPOID	INT IDENTITY (1,1)	PRIMARY KEY	Identificador único del equipo.
	TIPOEQUIPO	VARCHAR (20)	NOT NULL	Valores cortos como "LAPTOP", "CELULAR".
	MODELO	VARCHAR (30)	NOT NULL	Suficiente para códigos y nombres de modelos.

Tabla	Campo	Tipo de dato	Restricción	Justificación
	USUARIOID	INT	NOT NULL, FOREIGN KEY	Vincula el equipo con su propietario.
TECNICOS	TECNICOID	INT IDENTITY (1,1)	PRIMARY KEY	Identificador único del técnico.
	NOMBRE	VARCHAR (50)	NOT NULL	Nombre completo del profesional.
	ESPECIALIDAD	VARCHAR (30)	-	Área de conocimiento del técnico.
REPARACIONES	REPARACIONID	INT IDENTITY (1,1)	PRIMARY KEY	Identificador único del trámite.
	EQUIPOID	INT	NOT NULL, FOREIGN KEY	Vincula la reparación con el equipo afectado.
	FECHASOLICITUD	DATETIME	NOT NULL	Almacena fecha y hora exacta de la solicitud.
	ESTADO	VARCHAR (20)	-	Controla el ciclo de vida: PENDIENTE, EN PROCESO, FINALIZADO.
ASIGNACIONES	ASIGNACIONID	INT IDENTITY (1,1)	PRIMARY KEY	Identificador único de la asignación.

Tabla	Campo	Tipo de dato	Restricción	Justificación
	REPARACIONID	INT	NOT NULL, FOREIGN KEY	Vincula con la solicitud de reparación.
	TECNICOID	INT	NOT NULL, FOREIGN KEY	Vincula con el técnico responsable.
	FECHAASIGNACION	DATETIME	-	Fecha en que se asignó el trabajo.
DETALLEREPARACION	DETALLEID	INT IDENTITY (1,1)	PRIMARY KEY	Identificador único del detalle.
	REPARACIONID	INT	NOT NULL, FOREIGN KEY	Vincula con la solicitud principal.
	DESCRIPCION	VARCHAR (100)	-	Explicación de daños, trabajos realizados o repuestos.
	FECHAINICIO / FECHAFIN	DATETIME	-	Controla los tiempos de ejecución del trabajo.

2. Integridad Referencial: Llaves Primarias y Foráneas

Tabla	Llave Primaria (PK)	Llave Foránea (FK)	Referencia a (Tabla → Campo)
USUARIOS	USUARIOID	—	—
EQUIPOS	EQUIPOID	USUARIOID	USUARIOS → USUARIOID
TECNICOS	TECNICOID	—	—
REPARACIONES	REPARACIONID	EQUIPOID	EQUIPOS → EQUIPOID
ASIGNACIONES	ASIGNACIONID	REPARACIONID TECNICOID	REPARACIONES → REPARACIONID TECNICOS → TECNICOID
DETALLEREPARACION	DETALLEID	REPARACIONID	REPARACIONES → REPARACIONID

Integridad Referencial: Llaves Primarias y Foráneas

La integridad referencial permite mantener los datos organizados y conectados correctamente entre las tablas. La llave primaria (PK) identifica de manera única cada registro dentro de una tabla, mientras que la llave foránea (FK) permite relacionar una tabla con otra utilizando una llave primaria existente.

En este caso, primero se crean las tablas independientes como USUARIOS y TECNICOS, ya que no necesitan información de otras tablas. Después se crean EQUIPOS y REPARACIONES, porque dependen de los datos registrados anteriormente. Finalmente, se crean ASIGNACIONES y DETALLEREPARACION, que utilizan llaves foráneas para relacionarse con las reparaciones y técnicos.

Este orden es importante porque evita errores en las relaciones y asegura que la información de la base de datos sea correcta y consistente

Fase 2: Creación de la Estructura (DDL)

Script completo con comentarios y restricciones:

Primero realice una nueva consulta

CREATE DATABASE SISTEMAREPARACIONES (Cree una base de datos nueva)

USE SISTEMAREPARACIONES (Aquí le dije utilice esta tabla)

CREATE TABLE USUARIOS (Crea una tabla llamada USUARIOS para guardar la información de los clientes.

(

USUARIOID INT IDENTITY(1,1) PRIMARY KEY, (Cree un identificador único para cada usuario, como llave primaria.)

NOMBRE VARCHAR(50) NOT NULL, (Acá nombre del usuario y establece que este campo es obligatorio.)

CORREOELECTRONICO VARCHAR(100) NOT NULL, (Almacena el correo electrónico del usuario y no permite valores vacíos.)

TELEFONO VARCHAR(20), (Para guardar información del usuario.)

CONSTRAINT UQ_USUARIOS_CORREO UNIQUE(CORREOELECTRONICO)
(Use es comando para evitar que existan dos usuarios con el mismo correo electrónico.)

)

CREATE TABLE EQUIPOS (Cree la tabla donde se registran los equipos de los usuarios.)

(

EQUIPOID INT IDENTITY(1,1) PRIMARY KEY, (Lo utilice para crear un código único para identificar cada equipo.)

TIPOEQUIPO VARCHAR(20) NOT NULL, (Guarda el tipo de dispositivo, por ejemplo laptop o celular.)

MODELO VARCHAR(30) NOT NULL, (Lo use para guardar el modelo específico del equipo.)

USUARIOID INT NOT NULL, (Use este para ver a que usuario le pertenece el equipo.)

CONSTRAINT FK_EQUIPOS_USUARIOS (Aquí le asigne un nombre a la llave foránea)

FOREIGN KEY (USUARIOID) REFERENCES USUARIOS(USUARIOID) **(Aquí lo utilice para dar relación entre equipos y usuarios mediante la llave foránea.)**

)

CREATE TABLE TECNICOS **(Cree la tabla para almacenar la información de los técnicos.)**

(

TECNICOID INT IDENTITY(1,1) PRIMARY KEY, **(lo utilice para dar un identificador único para cada técnico.)**

NOMBRE VARCHAR(50) NOT NULL, **(Lo use para poner el nombre del técnico.)**

ESPECIALIDAD VARCHAR(30) **(lo utilice para ver la especialización del técnico.)**

)

CREATE TABLE REPARACIONES **(Cree esta tabla para registrar las solicitudes de reparación.)**

(

REPARACIONID INT IDENTITY(1,1) PRIMARY KEY, **(Lo utilice para identificar cada reparación con un número único.)**

EQUIPOID INT NOT NULL, Indica qué equipo necesita reparación.

FECHASOLICITUD DATETIME NOT NULL, **(Para gaurdar la fecha en la que se solicitó la reparación.)**

ESTADO VARCHAR (20), **(Para consultar el estado actual de la reparación.)**

CONSTRAINT FK_REPARACIONES_EQUIPOS **(Aquí le asigne un nombre a la llave foránea)**

FOREIGN KEY(EQUIPOID) REFERENCES EQUIPOS(EQUIPOID) **(Para relacionar cada reparación con un equipo registrado.)**

)

CREATE TABLE ASIGNACIONES **(Cree es tabla para asignar técnicos a las reparaciones.)**

(

ASIGNACIONID INT IDENTITY(1,1) PRIMARY KEY, **(Lo utilice para dar un identificador único para cada asignación.)**

REPARACIONID INT NOT NULL, **(Lo use para mostrar la reparación asignada)**

TECNICOID INT NOT NULL, (Lo use para mostrar cuál técnico es el encargado de realizar la reparación.)

FECHAASIGNACION DATETIME, (Aquí para saber la fecha en la que se asignó el técnico.)

CONSTRAINT FK_ASIGNACIONES_REPARACIONES (Aquí le asigne un nombre a la llave foránea)

FOREIGN KEY(REPARACIONID) REFERENCES (aquí para relacionar la asignación con una reparación existente.)

REPARACIONES(REPARACIONID),

CONSTRAINT FK_ASIGNACIONES_TECNICOS (Aquí le asigne un nombre a la llave foránea)

FOREIGN KEY(TECNICOID) REFERENCES TECNICOS(TECNICOID) (Aquí para relacionar la asignación con un técnico registrado.)

)

CREATE TABLE DETALLEREPARACION (Cree la tabla para guardar información detallada de cada reparación.)

(

DETALLEID INT IDENTITY(1,1) PRIMARY KEY, (Lo utilice para darle un identificador único al detalle de cada reparación.)

REPARACIONID INT NOT NULL, (Da la relación del detalle con la reparación específica.)

DESCRIPCION VARCHAR(100), (Aquí muestra la explicación del trabajo realizado.)

FECHAINICIO DATETIME, (Registra cuándo inició la reparación.)

FECHAFIN DATETIME, (Registra cuándo finalizó la reparación.)

CONSTRAINT FK_DETALLE_REPARACION TECNICOS (Aquí le asigne un nombre a la llave foránea)

FOREIGN KEY(REPARACIONID) REFERENCES
REPARACIONES(REPARACIONID) Aquí para conectar los detalles con la
reparación correspondiente.)

)

Insertar datos y realizar las pruebas correspondientes y sus 5 datos a cada tabla.

INSERT INTO DBO.USUARIOS

(NOMBRE, CORREOELECTRONICO, TELEFONO)

VALUES

('LUIS SALAZAR','LUISASR92@HOTMAIL.COM','89900635'),

('HELLEN RODRIGUEZ','HELLEN92@GMAIL.COM','86886984'),

('WENDY BARRANTES','WEN@YAHOO.NET','88601485'),

('ALONSO ALVARADO','ALONSO25@GMAIL.COM','73200159'),

('KEYLOR SALAZAR','KEYLORSA@GMAIL.COM','89987812');

INSERT INTO DBO.EQUIPOS

(TIPOEQUIPO, MODELO, USUARIOID)

VALUES

('LAPTOP','HP25510', 1),

('LAPTOP','APPLEMACBOOKPRO2', 2),

('LAPTOP','LENOVO82X700GCGJ', 3),

('CELULAR','HONOR8PRO2025', 4),

('TABLET','SAMSUNGS108825', 5);

INSERT INTO DBO.TECNICOS

(NOMBRE, ESPECIALIDAD)

VALUES

('JONATHAN RODRIGUEZ','REPARACION LAPTOPS'),

('ANA RICHMOND','MANTENIMIENTO PREVENTIVO'),

('JOSE JINESTA','REPARACION CELULARES'),

('ELENA ROJAS','EQUIPOS APPLE'),

('ERICK CARVAJAL','HARDWARE Y SOFTWARE');

```
INSERT INTO DBO.REPARACIONES  
(EQUIPOID, FECHASOLICITUD, ESTADO)
```

```
VALUES
```

```
(1,'2026-07-01','FINALIZADO'),  
(1,'2026-07-03','EN PROCESO'),  
(1,'2026-07-04','PENDIENTE'),  
(1,'2026-07-06','FINALIZADO'),  
(1,'2026-07-10','EN PROCESO');
```

```
INSERT INTO DBO.ASIGNACIONES  
(REPARACIONID, TECNICOID, FECHAASIGNACION)
```

```
VALUES
```

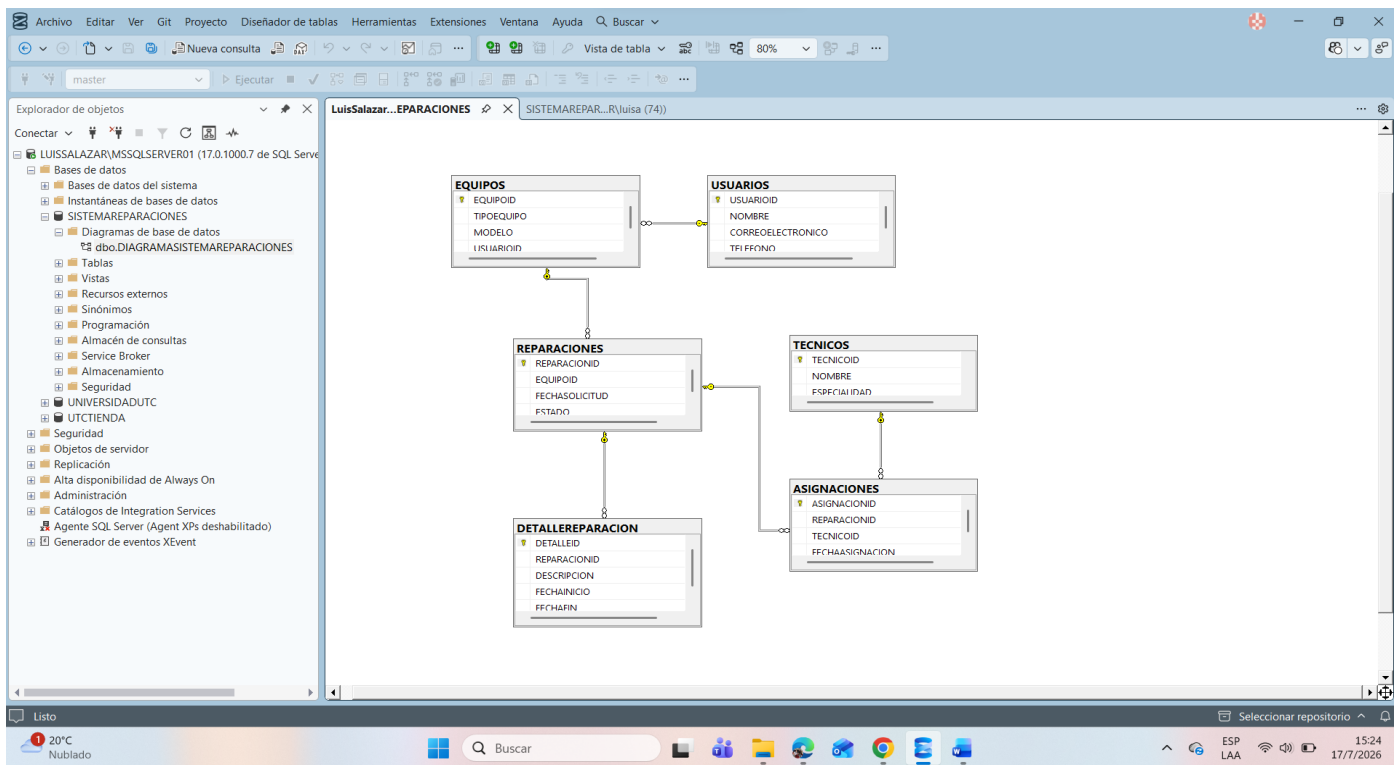
```
(1,1,'2026-07-01'),  
(2,4,'2026-07-03'),  
(3,4,'2026-07-04'),  
(4,3,'2026-07-06'),  
(5,5,'2026-07-10');
```

```
INSERT INTO DBO.DETALLEREPARACION  
(REPARACIONID, DESCRIPCION, FECHAINICIO, FECHAFIN)
```

```
VALUES
```

```
(1,'CAMBIO DE PANTALLA DAÑADA','2026-07-01','2026-07-04'),  
(2,'PROBLEMAS DE CONEXIONES','2026-07-03',NULL),  
(3,'PASTA TERMICA Y CAMBIO DE TARJETA','2026-07-04',NULL),  
(4,'BLOQUEADO POR INTRODUCIR PIN ERRONEO','2026-07-06','2026-07-07'),  
(5,'ACTUALIZACION DE SOFTWARE','2026-07-10',NULL);
```

Diagrama Sistema Reparaciones



Justificación Técnica

- Tipos de datos: Se eligieron VARCHAR para textos de longitud variable para no desperdiciar espacio, INT para identificadores por su rendimiento, y DATETIME para registrar fechas con precisión.
- Llaves foráneas: Se definieron respetando la relación de dependencia entre tablas; se evitó crear restricciones antes de que existan las tablas de referencia, garantizando que no se puedan eliminar usuarios o equipos que tengan reparaciones asociadas.
- Explicación desde mi proceso de diseño y construcción
- Al crear las relaciones entre las seis tablas, encontré que el principal problema era el orden en que se creaban. Si intentaba crear una tabla que dependía de otra que aún no existía, SQL mostraba un error porque la llave foránea no podía encontrar la tabla de referencia.
- Para solucionar este problema, primero creé las tablas independientes (USUARIOS y TECNICOS). Después creé las tablas que dependen de ellas (EQUIPOS y REPARACIONES). Finalmente, creé ASIGNACIONES y DETALLEREPARACION, ya que necesitan que las tablas anteriores ya existan. De esta manera, todas las llaves foráneas se crearon correctamente y la base de datos quedó organizada y sin errores.

Comandos para subir GitHub en mi repositorio:

Git Status = Para comprobar que la carpeta tenga archivos para subir.

Git add . = Para preparar los archivos en la carpeta.

Git commit -m = Para agregarle un comentario al archivo.

Git Push -u origin main = Para subir todos los archivos pendientes.

Git Status = Para verificar que todo se haya subido y no quede nada pendiente.

Conclusión

Este proyecto fue uno de los mayores retos que he enfrentado durante el curso. Al principio cometí varios errores relacionados con la lógica, el orden de creación de las tablas y las relaciones entre las llaves foráneas, lo que ocasionaba que el código no funcionara como esperaba. Sin embargo, cada error me ayudó a comprender mejor cómo funciona una base de datos y la importancia de planificar antes de programar.

Después de muchas horas de análisis, pruebas y correcciones, logré completar el proyecto correctamente. Ver que todo funcionaba al final me dio una gran satisfacción y me demostró que, con paciencia, práctica y dedicación, es posible superar cualquier dificultad.

Esta experiencia fue muy motivadora para mí, porque fortaleció mi confianza y me hizo ver que sí soy capaz de desarrollar proyectos cada vez más complejos. Sin duda, este trabajo me anima a seguir aprendiendo, aceptando nuevos retos y mejorando mis habilidades en programación y bases de datos.